BrickForge: Learning to Build Brick Models Under a Piece Budget

Yogya Mehrotra
Independent researcher
ymehrotra@ucsd.edu

November 2026

Abstract

Turning a 3D shape into a model made of interlocking toy bricks is a discrete design problem: every brick must sit on studs, nothing may overlap, the result must hold together, and a person must be able to assemble it in order. We study this problem under an explicit *piece budget* and ask how fidelity scales with the number of bricks allowed. We build a shared pipeline (voxelize, place, trim to one connected component, plan an assembly order, validate, and check stability under a simplified static force model) and compare three generators inside it: a hand-written greedy search, a 100k-parameter 3D convolutional policy trained by imitating that search, and the same policy fine-tuned with proximal policy optimisation on the fidelity of the finished build. On a frozen benchmark of 53 held-out cases (procedural shapes and Creative Commons Thingi10K meshes) at budgets of 50, 100 and 200 pieces, the learned pipeline reaches a mean intersection-over-union of 0.693 against the search's 0.705 at the 200-piece cap, ties it on procedural shapes, matches its validity and model-stability counts, and decodes in under 0.2 s per shape; a second training seed lands on the same number. Fidelity saturates between 100 and 200 pieces for every generator because the cap rarely binds. Three negative results are reported in full: the learned stop action stays miscalibrated on real meshes and is masked at decode time (a choice made on a held-out split, never on the benchmark); a stability term in the reward is redundant with the hard constraints; and the connectivity inputs matter where the policy judges on its own but not in the benchmark mean. All training ran in under five hours on one laptop. Code, benchmark records and an audit that regenerates every number are released.

1 Introduction

Give a child a 3D model of a pot and a box of bricks, and the question is not whether the pot can be approximated but how well it can be approximated with the bricks at hand. This paper is about that question for a computer: given a 3D shape and a limit on the number of bricks, produce a model that (a) looks like the shape, (b) is physically legal, in that every brick sits on studs and nothing overlaps, (c) holds together as a single structure, and (d) comes with an assembly order a person can follow. Figure 1 shows the task on one mesh.

The problem is interesting because it is *discrete* and *constrained*. Each decision is a choice among hundreds of legal placements, a wrong choice can block a cell forever, and whether the final structure is one connected piece or several loose islands only becomes clear at the end. Classical approaches search this space with hand-written heuristics. A natural question is whether a small learned policy can do as well, and whether the extra signal available from the finished build (its fidelity, its connectedness) can be used to train that policy beyond imitation. A second, practical question is how fidelity depends on the piece budget: a 50-piece model is cheaper to buy and faster to build than a 200-piece one, and a designer needs to know what is lost.

We answer both questions inside one controlled pipeline. The pipeline fixes the grid (16 studs by 16 studs by 8 brick layers), the catalogue (six basic bricks), the legality rules, the trim that keeps the largest ground-connected component, the assembly planner, the validator and a simplified static stability check. Three generators are swapped in and out of that pipeline:

- a **greedy search** that tiles each layer bottom-up, covering the most constrained cells first;
- a **learned policy**, a 3D convolutional network with about 100k parameters that scores every legal placement plus a stop action, trained by *supervised fine-tuning* (SFT) to imitate the search;

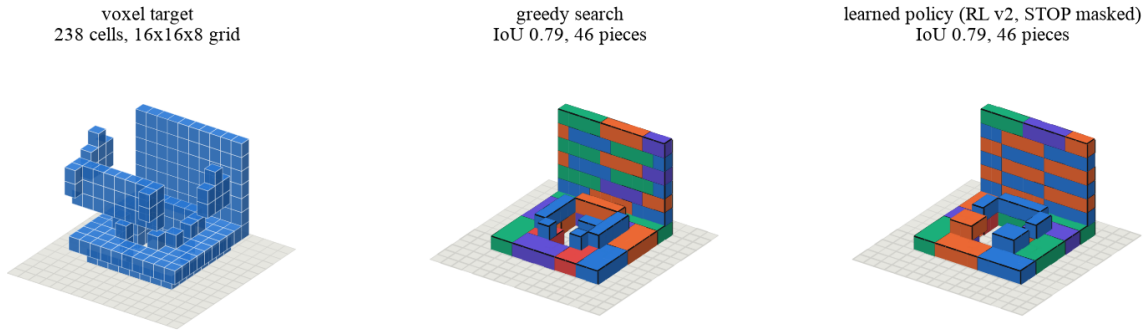


Figure 1: The task. A Thingi10K mesh is voxelized onto a $16 \times 16 \times 8$ grid (left) and rebuilt from six kinds of brick under a 200-piece budget by the hand-written greedy search (middle) and by the learned policy after reinforcement learning (right). Both builds pass the same validator and assembly planner. Brick colours are only there to separate neighbouring pieces.

- the same policy after **reinforcement learning** (RL) with proximal policy optimisation (PPO), rewarded only by the fidelity and model-stability of the finished, trimmed build.

Everything is evaluated on a frozen benchmark of 53 held-out cases (13 procedural shapes and 40 Creative Commons meshes from Thingi10K) at budgets of 50, 100 and 200 pieces, with an audit that regenerates every recorded number.

Findings. At the 200-piece cap the RL policy with its stop action masked scores a mean intersection-over-union (IoU) of 0.693 against 0.705 for the search; it ties the search on procedural shapes (0.826 vs 0.828), trails it by 0.015 on real meshes, matches its validity (51/53) and model-stability (49/53) counts, and decodes in about twice the search's time, both under 0.2 s per shape. Its head-to-head record against the search (wins / losses / ties) is 10 / 16 / 26; the median case is a tie and the gap sits in a handful of real meshes. A second training seed reproduces the number exactly. Each stage of the ladder earned its place: rebalancing the training data towards real meshes moved SFT from 0.614 to 0.662 (two-seed mean), RL on top added between 0.02 and 0.04 over its own initialisation in both seeds, and masking the stop action added another 0.02 to 0.03. Fidelity rises steeply from 50 to 100 pieces and barely from 100 to 200, for every generator, because the greedy and the policy both stop when no supported placement remains rather than at the cap.

What did not work. We report three negative results. The learned stop action remains miscalibrated on real meshes after RL: with its own stop the policy ends some builds at zero pieces, and masking the action gains 0.03 to 0.09 IoU on a held-out split. The stability bonus in the reward turned out to be redundant: without it the builds are exactly as stable (under the model) because the legality rules and the trim already enforce what the bonus rewards. And the connectivity inputs added for RL help wherever the policy has to judge on its own (its own stop decision, the larger held-out split) but are invisible in the benchmark mean once stop is masked and the trim cleans up after decoding.

Contributions.

1. A reproducible, audited piece-budget benchmark for brick building with full failure accounting (every case reported, failed builds scored zero).
2. A controlled ladder, greedy → SFT → SFT+RL on two datasets and two seeds, that isolates the effects of data, reinforcement learning and the decode rule.
3. Evidence that a 100k-parameter candidate-scoring policy trained in under five hours on a laptop reaches within 0.012 mean IoU of a hand-engineered search at comparable decode cost.
4. Three negative results about stop calibration, reward shaping and input features, each with the evidence that would have been needed to claim the opposite.

2 Problem setup and the shared pipeline

Grid and bricks. A brick model lives on an integer grid. The horizontal unit is one *stud* (8 mm pitch); the vertical unit is one *plate* (3.2 mm), and a standard brick is three plates tall. We use a 16×16 stud footprint and 8 brick layers, which

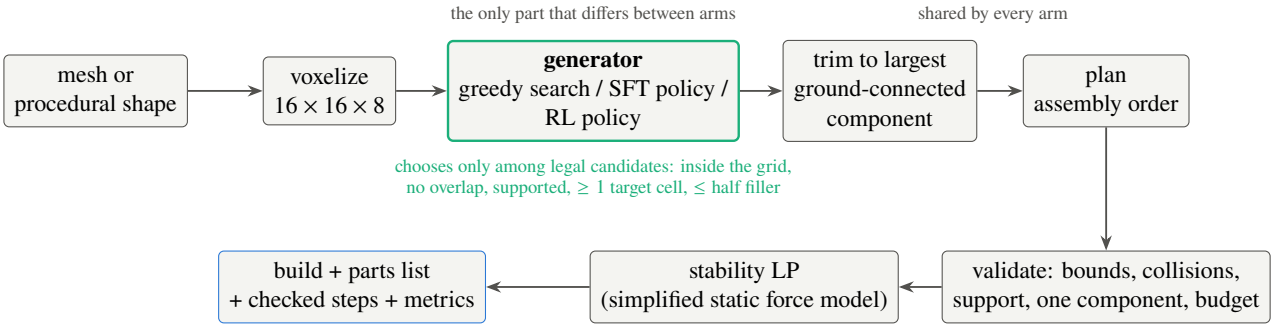


Figure 2: The shared pipeline. Generators differ only in how they choose the next brick; the trim, planner, validator and stability check are identical, so fidelity differences cannot come from different rules.

is 2048 cells of one stud by one stud by one brick. The catalogue has six upright bricks: 1×1, 1×2, 1×3, 1×4, 2×2 and 2×4, placed at 0° or 90°. A *placement* is a brick type, rotation and position; a *build* is a list of placements plus an ordered list of assembly steps.

Target. The input is a watertight mesh. It is rotated so its chosen axis points up, scaled uniformly to fit the physical box the grid represents, floored, and voxelized by testing whether each cell centre lies inside the mesh. The result is a boolean *target* grid T. (Procedural targets are generated directly on the grid.)

Legality. A placement is legal when it lies inside the grid, overlaps no placed brick, is *supported* (it sits on the ground or at least one cell directly beneath it is already filled), covers at least one target cell, and is at most half *filler*, cells outside the target. These rules are enforced by a candidate enumerator that every generator in this paper shares, so no generator can place an illegal brick.

The finalize step. Every generator returns a raw list of placements, and every list goes through the same four stages (Figure 2):

1. **Trim.** Bricks connect only through studs: an upper brick is attached to a lower one when it rests directly on it with overlapping footprints. Side contact does not count. The build is reduced to its largest component connected to the ground; everything else is discarded. This is what makes the output one object.
2. **Plan.** A planner orders the remaining bricks one per step, each inserted straight down, attached to what is already there, with a clear insertion path and never sealing an unplaced brick beneath it. For this catalogue of same-height upright bricks that greedy planner is complete: every valid build gets an order.
3. **Validate.** An independent validator replays the steps and checks bounds, collisions, support, the single-component property and the piece budget. A build that fails here is reported as invalid.
4. **Stability check.** A linear simplification of the StableLego force model [22]: bricks are rigid, loaded by their own weight; each shared stud cell carries vertical force at four points; compression is unbounded and tension is capped by a clutch capacity of 0.98 N per stud cell, divided over its four points. The constant is StableLego's measured value, which that work applies per contact point rather than per stud, so our limit is roughly four times more conservative than theirs. A linear program finds the smallest tension utilisation that balances every brick. Utilisation at most one is *stable under the model*. The model ignores lateral loads, friction and deformation, so throughout this paper "stable" means stable under the simplified static force model; not physically tested.

Metrics. *Fidelity* is the intersection-over-union between the occupancy of the trimmed build B and the target, IoU $= |B\cap T|/|B\cup T|$, so both missing and extraneous cells count. *Validity* is the validator's verdict, *stability* the force model's, *pieces* the brick count after the trim, and *decode time* the wall-clock seconds the generator took. The *piece budget* is a hard cap on placements; a generator may stop earlier.

Fidelity convention. Throughout, a reported mean IoU is the mean over the 52 benchmark cases that are attemptable (one fixture is a deliberately non-watertight mesh that every generator must reject), with a build that is missing or fails

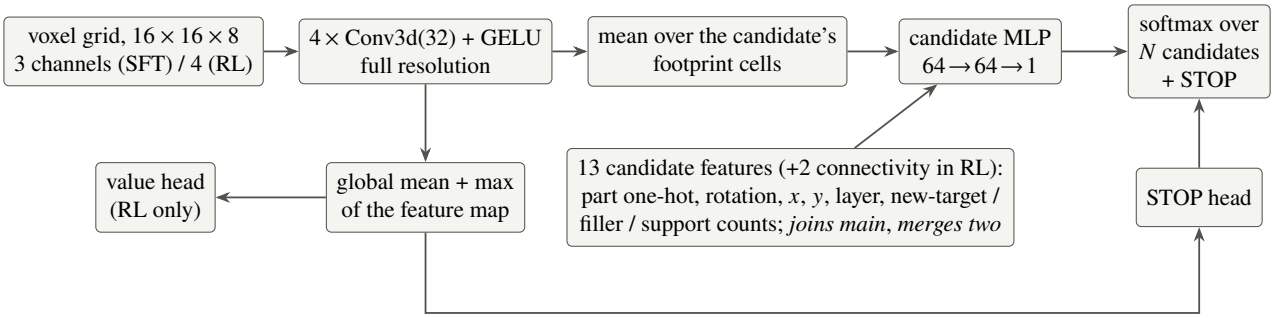


Figure 3: The candidate-scoring policy. Every legal placement is scored from pooled voxel features and a few hand-computed counts; STOP is scored from a global pool. 97k parameters (101k with the RL value head and connectivity inputs).

validation scored 0. A generator therefore cannot raise its mean by failing on hard cases. The benchmark's own summariser averages over valid builds only, which reads higher for generators with failures; we never use that number for comparisons.

3 Three ways to choose the next brick

3.1 Greedy search (the baseline and the teacher)

The baseline builds layer by layer from the ground up. Within a layer it repeatedly scores every legal placement of every brick type and takes the best. The score puts *hard* cells first: a target cell with nothing beneath it can only ever be covered by a brick that also overlaps the layer below, so the search covers those before they become unreachable. After hard cells it prefers placements that cover more new target cells, then fewer filler cells, then more support underneath; ties break on the number of distinct bricks below (to interlock), part cost and position. Three filler policies (none, at most a quarter, at most half) are run and the best validated IoU is kept. The whole search takes 0.07 s per shape on average. Its output, ordered by the planner, is also the *teacher* that the learned policy imitates.

3.2 The policy network

The learned generator is a candidate-scoring network (Figure 3). Its input is the grid as three voxel channels (target, current assembly, remaining target), encoded by four 3D convolutions with 32 channels at full resolution (the grid is small enough that no pooling is needed). For each legal placement the encoder's features are averaged over the placement's footprint cells and concatenated with 13 hand-computed features (the brick type as a one-hot code, rotation, position, and the counts of new target, filler and supporting cells); a small MLP turns that into one logit. A separate head scores the *stop* action from a global pool of the feature map. A softmax over all candidates plus stop gives the policy. Because only enumerated legal placements receive logits, legality is inherent at training and at inference. The policy has 97k parameters; RL adds a value head and two inputs described below, for 101k in total.

3.3 Imitation (SFT)

Data. Targets come from two sources. Ten procedural families (boxes, stacks, capped towers, bridges, polyominoes, ellipsoids, cylinders, stepped shapes, blobs, rows of arches), 200 instances each, grounded and face-connected. And a pool of Thingi10K meshes with every benchmark object excluded, voxelized by the production pipeline. The supervision for each target is the greedy search's validated build in planner order, ending with the stop action. We trained on two versions of this pool. **v1** kept 150 real meshes (at most 50k faces, and only those on which the teacher covered at least half the target), so training was 93 % procedural. **v2** kept every eligible mesh, 640 real meshes up to 200k faces, and draws half of each training batch from them. Examples are split 80/10/10 by object, with all symmetric copies and all files of one Thingiverse object in the same split.

Training. Teacher forcing with cross-entropy over candidates plus stop; AdamW at 10^{-3} with a cosine schedule, batch 32, 20,000 steps, 8-fold symmetry augmentation (rotations and reflections of the grid). Decisions are streamed by a

background producer because materialising every augmented decision would need about 40 GB. Checkpoints are selected by free-running decode IoU on a fixed validation subset, not by teacher-forced accuracy, because the latter hides compounding errors. One run takes 25 to 28 minutes on the laptop's GPU.

3.4 Reinforcement learning

Imitation reproduces the teacher's per-step choices but cannot see what only shows up in the finished build: whether the layers interlock into one component, or whether an early placement blocked a cell for good. RL scores the finished build directly.

Environment. An episode places bricks on one target with the same candidate set plus stop, under temperature-1 sampling. The state tracks a union-find over stud connections so the *main* (largest grounded) component is known at every step, exactly as the trim will decide it. Two things are added to the observation: a fourth voxel channel marking the main component, and two per-candidate features, *joins the main component* and *merges two components*. The SFT weights are zero-padded into this layout, so the initial RL policy produces the SFT policy's logits exactly. The terminal reward is computed after the shared finalize step:

$$r \;=\; \mathrm{IoU}_{\text{after trim}} \;+\; 0.1 \cdot [\text{stable under the model}],$$

with no per-step shaping. Validity cannot fail under the candidate rules, so no penalty is needed.

Algorithm. PPO [38] with a value head on the same encoder; advantage is reward minus value (terminal reward only), normalised per batch; clip 0.2, entropy bonus 0.01, a KL penalty of 0.05 towards the frozen initial policy, Adam at 10^{-4}, gradient clipping 1.0. Masked actions are removed from the entropy and KL terms before any multiplication; leaving them in poisons the backward pass with NaNs. Each iteration samples 128 episodes in 8 forked CPU workers and runs 3 epochs of updates on the GPU; 250 iterations take 1.6 to 2.4 hours. The best checkpoint is chosen by greedy-decode IoU on a fixed 64-example validation subset that mirrors the training mix (half real meshes in v2).

Decode rule. At inference the policy decodes greedily (argmax). The stop action can be used as learned, or *masked*, in which case decoding continues until no legal placement remains or the cap is reached, exactly as the greedy search stops. Section 4 describes how this choice was made.

4 Benchmark and protocol

The suite. 53 cases were frozen before any learned generator existed. Thirteen are procedural fixtures with known expected behaviour: simple solids (cube, slab, tower, L and T shapes, wedge), overhangs (pyramid, sphere), a budget-limited cylinder, a bridge, and three adversarial inputs (a flat sheet that cannot be covered, a one-cell pole that admits no valid build, and a non-watertight mesh that must be rejected). Forty are Thingi10K meshes [49], drawn deterministically from the models that are Creative Commons licensed and are closed, manifold, single-component solids with 100 to 200k faces, one file per Thingiverse object. The 40 meshes are the held-out set: every Thingiverse object in the benchmark is excluded from every training pool, and the procedural fixtures are hand-made shapes distinct from the ten training families. Licence, author and source link are recorded per mesh.

Records and audit. Every (case, generator, budget) triple produces one record with status, fidelity, coverage, piece count, validity, planning outcome, stability verdict, per-stage seconds, hardware and git commit. An *audit* command regenerates every non-failed record from scratch, validates it with the independent validator, recomputes IoU from occupancy and compares status, pieces, validity and IoU with what the file claims. Every benchmark file in this paper passed its audit, and the greedy rows are byte-identical to the frozen baseline file in every run. The audit earned its keep once: it exposed that one mesh voxelized to 288, 289 or 290 cells depending on the process, because the containment test re-checks ambiguous cell centres with a ray direction drawn from the global random generator. Voxelization now runs that test under a fixed, restored random state.

Table 1: Mean IoU after trim over the 52 attempted cases (failed builds scored 0) at each piece budget, with the split at the 200-piece cap, validity and model-stability counts out of 53, and mean decode seconds per case. SFT and RL v2 are the mean of two training seeds. STOP masked: decoding continues until no legal placement remains. Stable means stable under the simplified static force model; not physically tested.

generator	data	50	100	200	procedural	Thingi10K	valid	stable	s/case
greedy search (frozen baseline)	–	0.621	0.696	**0.705**	0.828	**0.668**	51	49	0.07
SFT	v1	0.523	0.602	0.614	0.812	0.554	49	46	0.12
SFT + RL	v1	0.563	0.643	0.654	0.826	0.603	49	45	0.13
SFT + RL, STOP masked	v1	0.593	0.676	0.688	0.826	0.646	51	48	0.14
SFT (2 s.)	v2	0.566	0.653	0.662	0.793	0.636	49	48	0.12
SFT + RL (2 s.)	v2	0.583	0.657	0.667	0.803	0.615	47	46	0.12
SFT + RL, STOP masked (2 s.)	v2	0.609	0.682	**0.693**	0.826	0.653	51	49	0.13

The split, validity, stability and time columns for the v2 rows are seed 0; seed 1 is in Table 2.

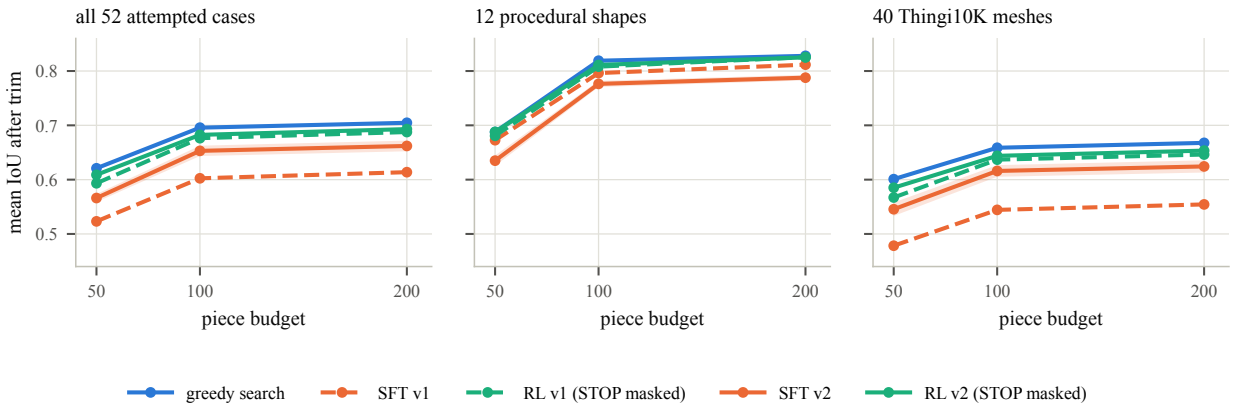


Figure 4: Fidelity against piece budget for the search (blue), imitation (orange) and reinforcement learning (green) on the v1 data (dashed) and the v2 data (solid). Bands are the range over two training seeds. Every arm gains most between 50 and 100 pieces and little between 100 and 200, because the cap rarely binds: the mean build has about 39 pieces at the 200 cap.

Budgets, seeds, cost. Every generator runs at budgets of 50, 100 and 200 pieces. Decode is deterministic (greedy), so the benchmark seed only matters for provenance; *training* seeds are reported separately (Section 5.4). Decode time is recorded per case on the same machine for every generator.

Decisions were made off the benchmark. The one decode choice for the learned generators, whether to use the learned stop or to mask it, was made on the SFT *test split* (260 held-out training-distribution examples, 188 procedural and 72 real), before any benchmark run of the affected checkpoint, and is reported in full in Section 5.5. The benchmark was never used to select a checkpoint, a decode rule or a hyperparameter.

5 Results

Table 1 gives the main numbers and Figure 4 the budget curves. We first read off the budget study, then the comparison between the learned policies and the search, then the ladder that attributes the gain, then seeds, ablations and failure analysis.

5.1 Piece budget versus fidelity

Fidelity is a concave function of the budget for every generator (Figure 4). From 50 to 100 pieces the search gains 0.075 and the best learned arm 0.074; from 100 to 200 they gain 0.009 and 0.012. The reason is visible in the records: at the 200 cap the mean build has about 39 pieces and the cap is the stopping reason for almost no case. Both the search and the policy stop when no supported, non-overlapping placement that covers a target cell remains, which for these shapes

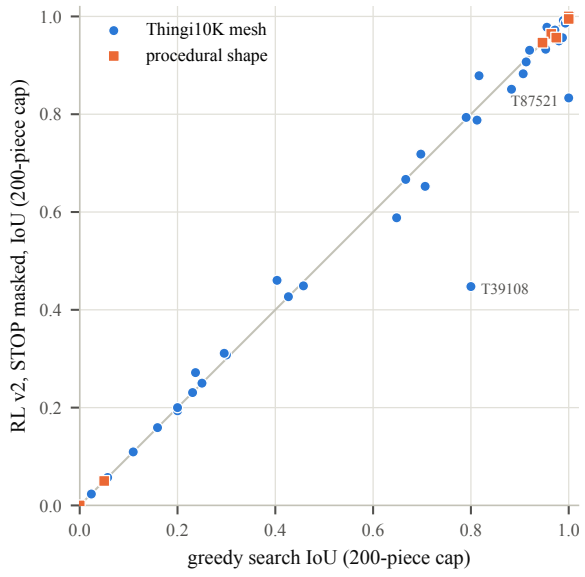


Figure 5: Per-case fidelity at the 200-piece cap, RL v2 with STOP masked against the greedy search. Most cases lie on the diagonal; the head-to-head record (wins / losses / ties) is 10 / 16 / 26. The honeycomb pot T39108 is the one large loss.

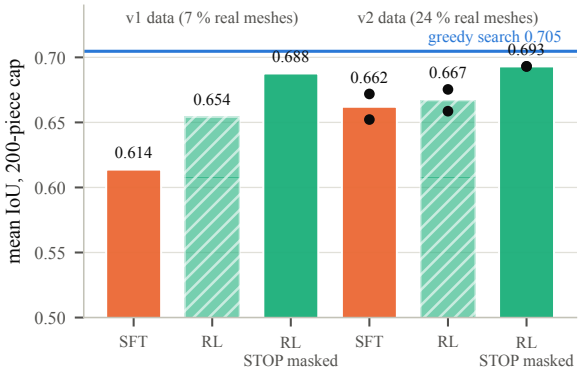


Figure 6: The ladder at the 200-piece cap. Hatched bars use the learned STOP; dots are the two training seeds of the v2 arms. Rebalancing the data moved SFT up, RL added more, and masking STOP added the rest.

happens long before 200 bricks. Fifty pieces, by contrast, truncates most real meshes mid-build. The practical reading for a designer is that on this grid a 100-piece budget buys nearly all the fidelity that 200 does. The ordering of the arms is the same at every budget, so the comparison below is reported at 200 without loss.

5.2 Learned policy versus hand-written search

The RL v2 policy with STOP masked is within 0.012 mean IoU of the search at every budget. On the 12 procedural shapes it ties (0.826 vs 0.828); on the 40 real meshes it trails by 0.015 (0.653 vs 0.668). Its validity count (51/53) and model-stability count (49/53) equal the search's; the two invalid cases are the adversarial fixtures that no generator can build, so 51 is the ceiling. It decodes in 0.13 s per case against the search's 0.07 s. Figure 5 shows the per-case picture: the median case is a tie, the head-to-head record is 10 / 16 / 26 (wins / losses / ties), and the remaining gap sits in a handful of real meshes, one of which (a honeycomb pot, Section 5.7) accounts for most of it.

Two comparisons bound what this means. Against its own SFT initialisation the RL policy's record is 26 / 3 / 23 (seed 0) and 27 / 9 / 16 (seed 1). Against a test-time-compute baseline, best-of-8 sampling from the v1 SFT policy at about seven times the decode cost, the single-sample RL v1 policy with STOP masked scores 0.688 against 0.686: RL's advantage over sampling is cost, not ceiling.

5.3 What produced the gain: data, RL, decode rule

Figure 6 separates the three changes between the first learned arm and the last.

- **Data.** The same SFT recipe on the v2 pool, which has four times as many real meshes and draws half of each batch from them, moves SFT from 0.614 to 0.662 (mean of two seeds, range 0.652–0.672). Given the 0.02 spread between seeds at the SFT stage, we state this gain as +0.04 to +0.06.
- **RL.** On the v1 data RL adds +0.040 over SFT with the learned stop (0.614 → 0.654). On the v2 data, with STOP masked, RL adds +0.021 to seed 0 and +0.041 to seed 1, and both seeds land on 0.693. RL beats its own initialisation 26 to 3 and 27 to 9 head to head. Where the gain comes from is shown by the diagnostic fields: on the v1 ladder, fragmented builds fall from 27 to 22 of 52 and the mean number of pieces removed by the trim falls from 6.0 to 3.2. Figure 7 shows the training curves: validation IoU rises from the SFT value to within 0.02 of the teacher on the same subset, fragmentation roughly halves, and policy entropy stays near 1.3 to 1.6 nats with the KL to the initial policy

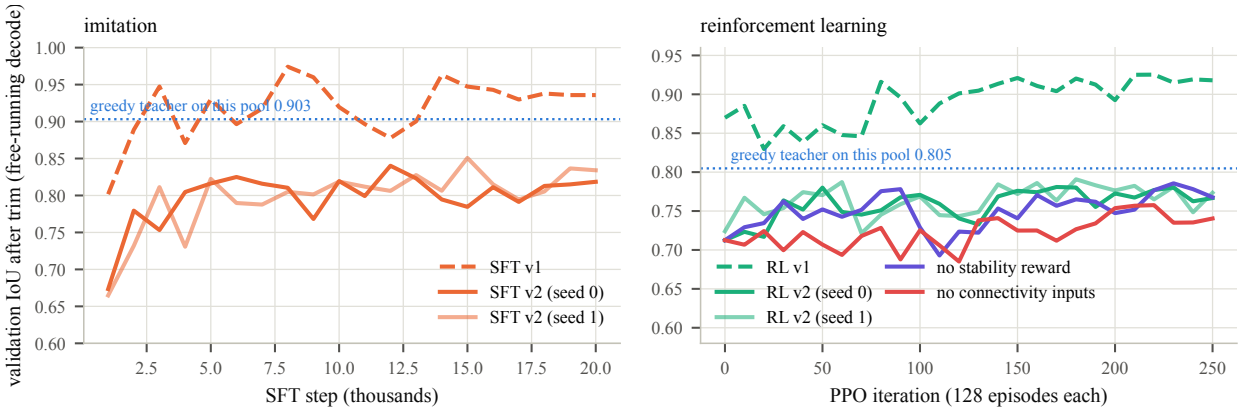


Figure 7: Training curves. Left: free-running validation IoU during imitation (every 1000 steps); the v1 pool is almost entirely procedural and its teacher reaches 0.903 on it, the v2 pool is half real meshes and harder. Right: validation IoU during PPO (every 10 iterations) for both v2 seeds, the v1 run and the two ablations; the dotted lines are the greedy teacher on the same validation subsets.

Table 2: The v2 ladder for both training seeds at the 200-piece cap (mean IoU over attempted cases; wins / losses / ties against the search). The v1 arms and the ablations are single-seed.

arm	s0	s1	procedural s0 / s1	Thingi10K s0 / s1	valid	stable	vs search s0; s1
SFT v2	0.672	0.652	0.793 / 0.783	0.636 / 0.613	49 / 48	48 / 44	4 / 26 / 22; 4 / 28 / 20
RL v2 (learned STOP)	0.659	0.675	0.803 / 0.803	0.615 / 0.637	47 / 48	46 / 46	7 / 27 / 18; 7 / 25 / 20
RL v2, STOP masked	0.693	0.693	0.826 / 0.824	0.653 / 0.654	51 / 51	49 / 47	10 / 16 / 26; 11 / 19 / 22

around 0.2 to 0.3, so the policy moved without collapsing.

- **Decode rule.** Masking the learned stop adds +0.034 to RL v1 and +0.034 (seed 0) / +0.018 (seed 1) to RL v2. One caveat belongs here: with its *own* stop, RL v2 (0.659) is below SFT v2 (0.672) on the mean, because the RL policy stops at zero pieces on a few real meshes; the RL gain is only realised once stop is masked (Section 5.5).

5.4 Two training seeds

Every arm of the v2 ladder was retrained end to end with a second seed (Table 2). The headline reproduces: RL v2 with STOP masked scores 0.693 and 0.693 at the 200 cap, 0.012 behind the search either way, and wins 10 and 11 cases against it. The per-case picture is also stable: the same 12 cases are lost to the search in both seeds and the same 6 real meshes are won in both. The one count where seed 1 is worse is stability, 47 against 49 stable builds at the 200 cap (validity is identical at 51). The SFT stage, by contrast, differs by 0.020 between seeds; RL removes that spread. Two seeds give a range, not a confidence interval, and we report both numbers rather than a mean with an error bar.

5.5 The stop action

Table 4 (Appendix C) is the decision table, computed on the SFT test split before each checkpoint touched the benchmark. Masking the stop action never costs anything on procedural shapes and gains 0.03 to 0.09 on real meshes for all five RL checkpoints (two seeds, two ablations, and v1), so the masked decode is the one reported for each. On the benchmark the learned stop is the cause of every zero-piece build: the policy stops before placing anything on three to six real meshes per checkpoint, different meshes for different seeds (seed 0: four meshes; seed 1: three; only one in common). Figure 8 shows how builds end. Our reading is that the policy learned that on shapes where every remaining candidate looks poor, stopping has the highest expected value under its own future behaviour, even though the terminal reward penalises exactly that and half the RL episodes were real meshes. Masked decoding forces the policy to keep tiling and lets the trim keep the largest connected part; it fragments more often (25 of 40 real builds have pieces removed by the trim, against 14 with the learned stop) but ends with more of the target covered.

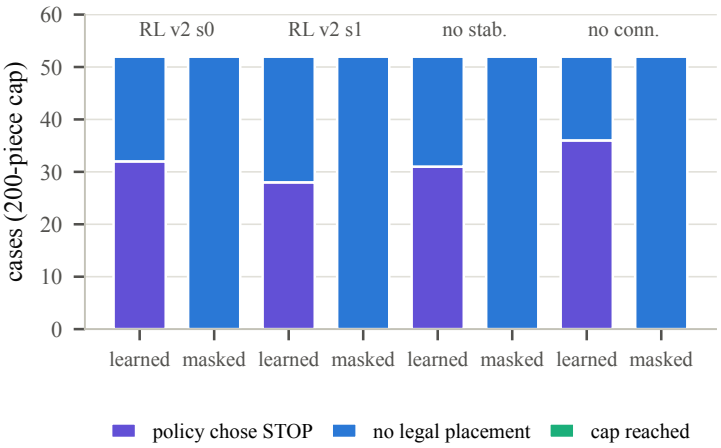


Figure 8: Why builds end at the 200-piece cap, for each RL checkpoint with the learned and the masked stop. The cap itself is never reached. With the learned stop, the policy chooses to stop on 28 to 36 of 52 cases, including a few at zero pieces.

Table 3: Ablations of RL v2 at the 200-piece cap (single seed; STOP masked unless stated). Wins / losses / ties are against the full RL v2 with STOP masked. Stable means stable under the simplified static force model; not physically tested.

arm	mean IoU	procedural	Thingi10K	valid	stable	vs RL v2
RL v2, STOP masked (reference)	0.693	0.826	0.653	51	49	–
no stability reward	0.686	0.823	0.645	51	49	7 / 16 / 29
no connectivity inputs	0.690	0.824	0.650	51	49	7 / 19 / 26
no stability reward, learned STOP	0.644	0.789	0.601	46	46	
no connectivity inputs, learned STOP	0.595	0.792	0.536	45	45	

5.6 Ablations

Two arms were retrained from the same seed-0 SFT checkpoint with the RL v2 settings, changing one thing each (Table 3).

The stability reward is redundant. With the stability term removed from the reward, the builds are exactly as stable as with it: 147 of 153 valid builds across the three budgets, the same two real meshes unstable at the 200 cap, identical to the search. The legality rule that every brick must rest on something, together with the trim to one grounded component, already produces structures the force model accepts; the bonus had nothing left to reward. Fidelity moves by 0.007 (7 / 16 / 29 head to head), a single-seed difference smaller than the 0.02 seed spread at the SFT stage, though one case moves a lot (the honeycomb pot falls from 0.45 to 0.25). The clean statement is that in this setting stability comes from the hard constraints, not from reward shaping.

The connectivity inputs matter where the policy judges on its own. Removing the main-component channel and the two connectivity features costs 0.064 on the benchmark under the learned stop (0.595 vs 0.659), with six real meshes ending at zero pieces; it costs 0.035 on the 260-example test split with stop masked (Table 4 in Appendix C, 0.805 vs 0.840) and 0.023 on the validation curve. Yet on the 52-case benchmark with stop masked the means are within 0.003 (0.690 vs 0.693), because one large win on the honeycomb pot (0.78 vs 0.45) offsets 18 losses to 7 wins. Head to head it is the weaker policy; the benchmark mean is too coarse to show it. The features' main job is telling the policy which candidates attach to the main component and when a build is finished. With stop masked, the trim does that cleanup after decoding for free.

5.7 Failure analysis

Figure 9 shows where the remaining gap lives. Twelve cases are lost to the search in both seeds: two procedural shapes by at most 0.02 (sphere, wedge) and ten real meshes. One shape class dominates: thin-walled shells with interior structure.

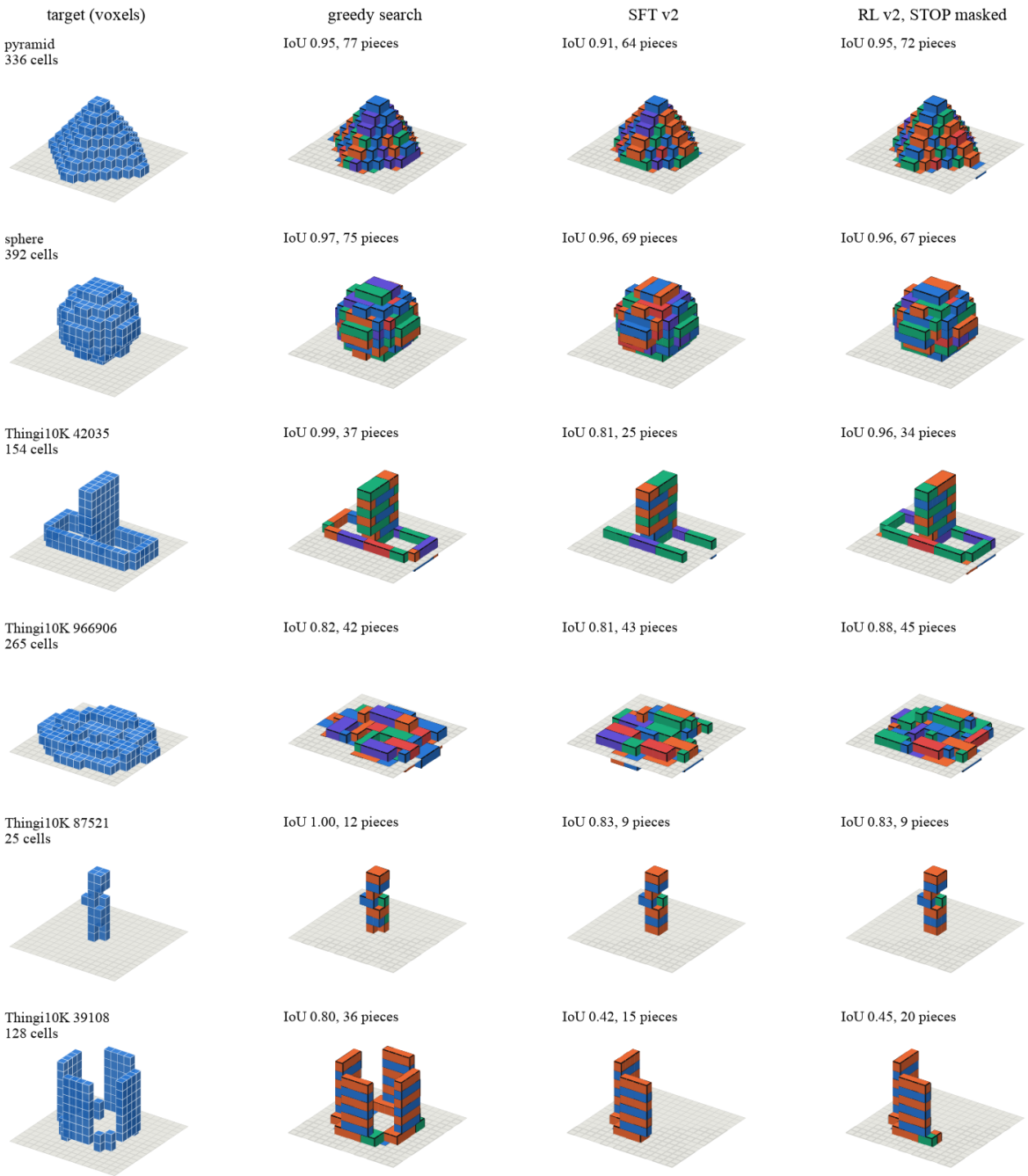


Figure 9: Six benchmark cases at the 200-piece cap: the voxel target, the search, SFT v2 and RL v2 with STOP masked. Rows: two procedural shapes (the pyramid shows SFT dropping pieces that RL recovers), a mesh where RL recovers most of SFT's loss (T42035), the largest RL win (T966906), a small mesh lost by both learned arms (T87521), and the honeycomb pot (T39108), the one case any learned generator loses by more than 0.35. Colours separate neighbouring bricks and carry no meaning.

The honeycomb pot (T39108) is the only case any learned generator loses by more than 0.35 (search 0.80, RL 0.45 and 0.42 across seeds): the policy tiles the outer walls as it would a solid and 19 of its 39 pieces are removed by the trim as unconnected. The search's exhaustive scan of every placement at every step, with hard cells first, still beats a learned preference on such shapes. The six real meshes won in both seeds are the opposite: broad, low shapes where the policy's placements interlock better than the search's layer-wise tiling and the trim removes less. Finally, the two cases that are valid but unstable under the model for every generator are tiny fragments (3 to 5 pieces with overhangs) left after trimming a mesh that occupies fewer than 90 of the 2048 cells; grid resolution, not the generator, is the limit there.

6 Negative results and limitations

We state these as findings rather than caveats, because each one changes what the headline means.

- **Learned stopping is unsolved.** After imitation, after RL on a half-real-mesh episode mix, and in both ablations, the policy's own stop action ends some real-mesh builds at zero pieces. The reported numbers use a masked stop and thereby borrow the search's stopping rule. A policy that knows when it is done remains future work.
- **The stability bonus did nothing.** Stability under the model is produced by the legality rules and the trim; the reward term is redundant here. We did not test harder settings (plates, overhanging parts, larger grids) where it might matter.
- **The connectivity features' benefit is only visible off the benchmark mean.** Their effect is clear in the policy's own stopping, on the larger held-out split and head to head, but a reader who looks only at Table 3's mean would conclude they are useless. Fifty-two cases are too few to resolve a 0.02 effect when one case can move by 0.3.
- **Stability is a model, not a test.** The force model uses a literature clutch constant, ignores lateral loads, friction and deformation, and no build was assembled physically. Every "stable" in this paper means stable under the simplified static force model; not physically tested. It is also conservative in two ways relative to StableLego [22]: the capacity is divided over four points per stud rather than applied per contact point, and equilibrium is a hard constraint, a formulation StableLego found solvable on only 58 % of its physical samples. Some builds we call unstable would likely stand.
- **The learned policies are distilled from the search.** "Learned versus hand-engineered" is really learned-from-hand-engineered: the imitation targets are the search's builds, and RL starts from that policy with a KL penalty towards it. The policy did not discover a different style of building; it got better at the search's.
- **Small benchmark, few seeds.** Fifty-two attempted cases, two training seeds for the v2 ladder, one seed for the v1 arms and the ablations, one decode seed (decoding is deterministic). Every reporting guideline we know of asks for more seeds than this [1, 11, 15]. Differences below about 0.02 should be read as observed, not established. We report ranges, not confidence intervals.
- **A small world.** One grid ($16 \times 16 \times 8$), solid fill, six upright bricks, one colour, no plates or slopes, no inventory constraints; recent learned generators use $20 \times 20 \times 20$ grids and eight brick types [2, 34]. Thin objects occupy 20 to 90 of 2048 cells and fragment at this resolution; grid resolution is the main lever and was not varied.
- **Three budget points, and the largest rarely binds.** The budget study has three points, and at 200 the cap is almost never the stopping reason. A finer sweep below 100 pieces would say more about the trade-off.
- **Comparable, not equal, decode cost.** The policy takes about twice the search's time per shape, both under 0.2 s. Best-of-8 sampling from the SFT policy nearly matches RL at about seven times the cost, so RL's advantage is cost, not a higher ceiling.
- **Thingi10K has no semantic categories.** The 40 meshes are a deterministic draw from the licensed, watertight subset, not a curated set of recognisable objects, and one benchmark is not a general claim.

7 Related work

Legolization by optimisation. Turning a shape into a brick layout has been posed as a layout optimisation problem for over a decade: graph-connectivity heuristics that find and repair weak points [43], force-based stability with layout refinement [24], maximum-flow stability bounds [44], genetic algorithms over brick layouts [19, 20], automatic assembly manuals [28], and extensions to Technic structures [48]. Our greedy search inherits this line's objectives (cover the target, interlock layers, use few pieces) in a deliberately simple form. It is a frozen baseline and a teacher, not a contribution.

Learned brick generation. Sequential assembly was first learned with Bayesian optimisation [17] and then with reinforcement learning: Brick-by-Brick [9] trains a PPO policy with a learned validity mask and a per-step IoU reward,

ending each episode when a per-task brick budget is spent, with no stop action. BrECS [2] replaces RL with a 3D-convolutional placement scorer behind a constraint filter, trained from a procedural teacher under explicit per-type brick budgets, and reports that this beats the RL and generative baselines; it is the closest relative of our SFT arm. Image- and text-conditioned generators have since adopted a stability oracle: BrickGPT [34] uses StableLego for rejection sampling and rollback, a rollback-free successor trains with physics rewards after SFT and reports that the RL stage also cuts brick counts [47], and BrickAnything [26], BrickNet [18], LEGO-Maker [12], Image2Lego [21] and ResemBrick [7] cover geometry, graph, image and photograph inputs. In the works we could verify, the budgeted learners carry no physics and the physics-aware generators carry no budget; we found none that combines an explicit piece budget with a force-based stability check in a learned policy, which is the gap this paper's setup sits in. Recent generators work on $20 \times 20 \times 20$ grids with eight brick types; our $16 \times 16 \times 8$ grid and six bricks are smaller than every setting we found.

Stability and assembly planning. Our force model is a linear simplification of StableLego [22], which descends from the per-brick equilibrium of Luo et al. [24]. StableLego caps the friction force at 0.98 N *per contact point*, places the equilibrium residuals in the objective so every structure is solvable, and reports 99.6 % agreement with 225 physically built samples; it also reports that a hard-constrained equilibrium formulation was solvable on only 58 % of those samples. Our program keeps hard equilibrium constraints and divides 0.98 N over the four points of each stud cell, so it is more conservative on both counts, and an infeasible program is reported as a topple. BrickSim [45] and BrickFEM [32] are the higher-fidelity alternatives. For assembly order, Liu et al. [23] train a PPO planner behind a data-free action mask that includes a per-step StableLego check and reach 100 % physical success where a connectivity-only mask reaches 30 to 93 %. Our planner is rule-based and complete for upright same-height bricks (Appendix E); stability is checked on the final build, with a per-step option that the benchmark does not use.

Imitation, then reinforcement learning. Behaviour cloning [33] and its compounding-error analysis [36] underlie the SFT stage; learning a policy from a search procedure's own choices is the AlphaGo recipe [3, 39, 40]. Fine-tuning with PPO [38] under a KL penalty towards the initial policy follows Ziegler et al. [50] and Ouyang et al. [29]; the KL term is what prevents forgetting in VPT [4], and PIRLNav [35] reports that the gain from RL fine-tuning shrinks as the imitation data grows, consistent with our +0.02 to +0.04. Rudner et al. [37] describe pathologies of KL-regularised RL with parametric references. Legal-action masking as a differentiable logit transform is analysed by Huang and Ontañón [16]; our candidate enumeration makes masking structural rather than a transform on a fixed action space, as in the object-centric policies of Bapst et al. [5] and Ghasemipour et al. [13]. None of the construction policies above learns a stop action: episodes end on a budget, a success test or a step cap. The nearest analyses of learned termination come from elsewhere: option termination collapses without a deliberation cost [14], navigation agents fail to stop unless stopping has its own head [46], and budget-induced terminations should not be treated as true terminal states [30]. We found no prior study of a stop action that is trained but masked at inference.

Evaluation. Thingi10K [49] supplies our real meshes; we found no brick-generation paper that uses it (the field evaluates on ModelNet and ShapeNet derivatives). Occupancy is computed by parity testing [27]; volumetric IoU follows the reconstruction literature [8, 25], whose critics note that IoU rewards coarse agreement [42]. Scoring a failed build as zero is stricter than reporting over valid structures only, as BrickGPT does. Best-of-N sampling as a test-time-compute baseline follows Cobbe et al. [10], Brown et al. [6] and Snell et al. [41]. On seeds, every guideline we know asks for more than two [1, 11, 15]; we therefore report ranges and make no significance claims, and follow the reproducibility checklist of Pineau et al. [31] in releasing the records and the audit.

8 Conclusion

Inside one fixed pipeline of legality rules, a connected-component trim, an assembly planner, a validator and a simplified stability model, a 100k-parameter candidate-scoring policy trained for under five hours on a laptop builds brick models within 0.012 mean IoU of a hand-engineered search at comparable decode cost, with identical validity and model-stability counts, reproduced across two training seeds. Fidelity saturates between 100 and 200 pieces for every generator because the cap rarely binds. Reinforcement learning on the fidelity of the finished build supplied most of the gain beyond imitation, and rebalancing the data towards real meshes supplied the rest; the stability reward supplied nothing, because the hard constraints already enforce it. The policy's own stop action remains the open problem, and thin-walled shells

remain the search's. The benchmark, its audit, the training recipes and the figure scripts are released so that every number here can be regenerated.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2108.13264.

[2] Seokjun Ahn, Jungtaek Kim, Minsu Cho, and Jaesik Park. Budget-aware sequential brick assembly with efficient constraint satisfaction. *Transactions on Machine Learning Research*, 2024. arXiv:2210.01021.

[3] Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search. *arXiv preprint arXiv:1705.08439*, 2017.

[4] Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video PreTraining (VPT): Learning to act by watching unlabeled online videos. *arXiv preprint arXiv:2206.11795*, 2022.

[5] Victor Bapst, Alvaro Sanchez-Gonzalez, Carl Doersch, Kimberly L. Stachenfeld, Pushmeet Kohli, Peter W. Battaglia, and Jessica B. Hamrick. Structured agents for physical construction. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019. arXiv:1904.03177.

[6] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

[7] Xilun Chen, Hanwen Wan, Yusong Zhao, Zexin Lin, Ruixiang Liao, and Xiaoqiang Ji. ResemBrick: Brick reconstruction from photographs with perceptual fidelity and buildability. *arXiv preprint arXiv:2608.09597*, 2026.

[8] Christopher B. Choy, Danfei Xu, JunYoung Gwak, Kevin Chen, and Silvio Savarese. 3D-R2N2: A unified approach for single and multi-view 3D object reconstruction. *arXiv preprint arXiv:1604.00449*, 2016.

[9] Hyunsoo Chung, Jungtaek Kim, Boris Knyazev, Jinhwi Lee, Graham W. Taylor, Jaesik Park, and Minsu Cho. Brick-by-brick: Combinatorial construction with deep reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2110.15481.

[10] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[11] Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. How many random seeds? Statistical power analysis in deep reinforcement learning experiments. *arXiv preprint arXiv:1806.08295*, 2018.

[12] Jiahao Ge, Mingjun Zhou, Hanyou Zheng, Hao Xu, and Chi-Wing Fu. LEGO-maker: Autoregressive image-conditioned LEGO model creation. *ACM Transactions on Graphics*, 44(6), 2025. doi: 10.1145/3763285.

[13] Seyed Kamyar Seyed Ghasemipour, Daniel Freeman, Byron David, Shixiang Shane Gu, Satoshi Kataoka, and Igor Mordatch. Blocks assemble! Learning to assemble with large-scale structured reinforcement learning. *arXiv preprint arXiv:2203.13733*, 2022.

[14] Jean Harb, Pierre-Luc Bacon, Martin Klissarov, and Doina Precup. When waiting is not an option: Learning options with a deliberation cost. *arXiv preprint arXiv:1709.04571*, 2017.

[15] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018. arXiv:1709.06560.

[16] Shengyi Huang and Santiago Ontañón. A closer look at invalid action masking in policy gradient algorithms. In *Proceedings of the International FLAIRS Conference*, volume 35, 2022. arXiv:2006.14171.

[17] Jungtaek Kim, Hyunsoo Chung, Jinhwi Lee, Minsu Cho, and Jaesik Park. Combinatorial 3D shape generation via sequential assembly. In *NeurIPS 2020 Workshop on Machine Learning for Engineering Modeling, Simulation, and Design*, 2020. arXiv:2004.07414.

[18] Peter Kulits and Cordelia Schmid. BrickNet: Graph-backed generative brick assembly. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. arXiv:2604.22984.

[19] Sangyeop Lee, Jinhyun Kim, Jae Woo Kim, and Byung-Ro Moon. Finding an optimal LEGO brick layout of voxelized 3D object using a genetic algorithm. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2015. doi: 10.1145/2739480.2754667.

[20] Seung-Mok Lee, Jae Woo Kim, and Hyun Myung. Split-and-merge-based genetic algorithm (SM-GA) for LEGO brick sculpture optimization. *IEEE Access*, 6:40429–40438, 2018. doi: 10.1109/ACCESS.2018.2859039.

[21] Kyle Lennon, Katharina Fransen, Alexander O'Brien, Yumeng Cao, Matthew Beveridge, Yamin Arefeen, Nikhil Singh, and Iddo Drori. Image2Lego: Customized LEGO set generation from images. *arXiv preprint arXiv:2108.08477*, 2021.

[22] Ruixuan Liu, Kangle Deng, Ziwei Wang, and Changliu Liu. StableLego: Stability analysis of block stacking assembly. *IEEE Robotics and Automation Letters*, 9(11):9383–9390, 2024. doi: 10.1109/LRA.2024.3414281. arXiv:2402.10711.

[23] Ruixuan Liu, Alan Chen, Weiye Zhao, and Changliu Liu. Physics-aware combinatorial assembly sequence planning using data-free action masking. *IEEE Robotics and Automation Letters*, 10(5):4882–4889, 2025. doi: 10.1109/LRA.2025.3555074. arXiv:2408.10162.

[24] Sheng-Jie Luo, Yonghao Yue, Chun-Kai Huang, Yu-Huan Chung, Sei Imai, Tomoyuki Nishita, and Bing-Yu Chen. Legolization: Optimizing LEGO designs. *ACM Transactions on Graphics*, 34(6):222:1–222:12, 2015. doi: 10.1145/2816795.2818091.

[25] Lars Mescheder, Michael Oechsle, Michael Niemeyer, Sebastian Nowozin, and Andreas Geiger. Occupancy networks: Learning 3D reconstruction in function space. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. arXiv:1812.03828.

[26] Zhengyang Ni, Feng Yan, Yu Guo, and Fei Wang. BrickAnything: Geometry-conditioned buildable brick generation with structure-aware tokenization. *arXiv preprint arXiv:2605.26182*, 2026.

[27] Fakir S. Nooruddin and Greg Turk. Simplification and repair of polygonal models using volumetric techniques. *IEEE Transactions on Visualization and Computer Graphics*, 9(2):191–205, 2003. doi: 10.1109/TVCG.2003.1196006.

[28] Sumiaki Ono, Alexis André, Youngha Chang, and Masayuki Nakajima. LEGO builder: Automatic generation of LEGO assembly manual from 3D polygon model. *ITE Transactions on Media Technology and Applications*, 1(4):354–360, 2013. doi: 10.3169/mta.1.354.

[29] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.

[30] Fabio Pardo, Arash Tavakoli, Vitaly Levdik, and Petar Kormushev. Time limits in reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *PMLR*, pages 4042–4051, 2018. arXiv:1712.00378.

[31] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 2021. arXiv:2003.12206.

[32] Martin Pletz and Matthias Drvoderic. BrickFEM: An automated finite element model for static and dynamic simulations of simple Lego sets. engrXiv preprint, 2023.

[33] Dean A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems 1*, 1989.

[34] Ava Pun, Kangle Deng, Ruixuan Liu, Deva Ramanan, Changliu Liu, and Jun-Yan Zhu. Generating physically stable and buildable brick structures from text. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 14798–14809, 2025. arXiv:2505.05469.

[35] Ram Ramrakhya, Dhruv Batra, Erik Wijmans, and Abhishek Das. PIRLNav: Pretraining with imitation and RL finetuning for ObjectNav. *arXiv preprint arXiv:2301.07302*, 2023.

[36] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011. arXiv:1011.0686.

[37] Tim G. J. Rudner, Cong Lu, Michael A. Osborne, Yarin Gal, and Yee Whye Teh. On pathologies in KL-regularized reinforcement learning from expert demonstrations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2212.13936.

[38] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[39] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. doi: 10.1038/nature16961.

[40] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550 (7676):354–359, 2017. doi: 10.1038/nature24270.

[41] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[42] Maxim Tatarchenko, Stephan R. Richter, René Ranftl, Zhuwen Li, Vladlen Koltun, and Thomas Brox. What do single-view 3D reconstruction networks learn? *arXiv preprint arXiv:1905.03678*, 2019.

[43] Romain Testuz, Yuliy Schwartzburg, and Mark Pauly. Automatic generation of constructable brick sculptures. In *Eurographics 2013 – Short Papers*, pages 81–84, 2013. doi: 10.2312/conf/EG2013/short/081-084.

[44] Martin Waßmann and Karsten Weicker. Maximum flow networks for stability analysis of LEGO structures. In *Algorithms – ESA 2012*, volume 7501 of *LNCS*, pages 813–824, 2012. doi: 10.1007/978-3-642-33090-2_70.

[45] Haowei Wen, Ruixuan Liu, Weiyi Piao, Siyu Li, and Changliu Liu. BrickSim: A physics-based simulator for manipulating interlocking brick assemblies. *arXiv preprint arXiv:2603.16853*, 2026.

[46] Jiannan Xiang, Xin Eric Wang, and William Yang Wang. Learning to stop: A simple yet effective approach to urban vision-language navigation. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020. arXiv:2009.13112.

[47] Chenhui Xu, Ziyue Bai, Fuxun Yu, Heng Huang, and Jinjun Xiong. Rollback-free stable brick structures generation. *arXiv preprint arXiv:2605.06947*, 2026.

[48] Hao Xu, Ka-Hei Hui, Chi-Wing Fu, and Hao Zhang. Computational LEGO technic design. *ACM Transactions on Graphics*, 38(6):196:1–196:15, 2019. doi: 10.1145/3355089.3356504.

[49] Qingnan Zhou and Alec Jacobson. Thingi10K: A dataset of 10,000 3D-printing models. *arXiv preprint arXiv:1605.04797*, 2016.

[50] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.

A Benchmark suite

Procedural fixtures (13). cube ($8 \times 8 \times 4$ grid), slab, tower, L shape, T shape, wedge (simple solids, expected valid with coverage at least 0.8); pyramid, sphere (overhangs); cylinder (budget-limited, coverage at least 0.5); bridge; flat sheet (cannot be covered, expected partial); thin pole (zero valid builds, expected no valid candidate); open box (non-watertight, expected rejected at parse time). The last is the one case excluded from every mean; the thin pole is attempted and scores 0 for every generator.

Thingi10K subset (40). Drawn with seed 0 from the Hugging Face mirror of Thingi10K among models that are CC BY, CC BY-SA or public domain, closed, manifold, single-component solids with 100 to 200k faces, one file per Thingiverse object, all with $+z$ up. Licence, author, name and source link are stored per model in the benchmark records. The meshes are downloaded by a script and never redistributed.

B Hyperparameters and cost

Policy	$4 \times$ Conv3d(3^3, 32 ch) + GELU; candidate MLP 64-64-1 on pooled features + 13 (15 in RL) candidate features; STOP and value MLPs 64-1 on global mean + max; 97,058 parameters (101,283 with the RL layout and value head)
SFT	teacher forcing, cross-entropy over candidates + STOP; AdamW 10^{-3}, cosine schedule, batch 32, 20,000 steps, grad clip; 8-fold symmetry augmentation; streaming item pool; free-run eval every 1000 steps on 64 examples; v2 draws half of each pool refresh from real meshes and drops examples whose teacher covers under half the target
RL	PPO, clip 0.2, entropy 0.01, KL 0.05 to the frozen initial policy, value coefficient 0.5, Adam 10^{-4}, grad clip 1.0, $\gamma = 1$ (terminal reward), advantages normalised per batch; 250 iterations $\times$ 128 episodes, temperature 1, 8 forked CPU rollout workers, 3 epochs, minibatch 128; reward IoU after trim + $0.1 \cdot$ stable; eval every 10 iterations on 64 examples (half real in v2); best checkpoint by eval IoU
Data	v1: 2000 procedural + 150 real (at most 50k faces, teacher coverage at least 0.5), split 1750 / 194 / 206. v2: 2000 procedural + 640 real (at most 200k faces, weak-teacher targets kept with their coverage recorded), split 2132 / 248 / 260 (test: 188 procedural, 72 real of which 30 weak-teacher)
Hardware	one Apple M5 Pro laptop, 24 GB; SFT and PPO updates on Metal, rollouts and all benchmark decoding on CPU for bit-reproducibility
Wall clock	SFT 23 to 28 min per run; RL v1 2 h 25 min, RL v2 1 h 35 min (seed 0) and 2 h 22 min (seed 1), ablations about 2 h 25 min each; the frozen benchmark 60 s for the search and a few minutes per learned generator; the audit regenerates every record in a comparable time

C The decode decision

Table 4: The decode decision. Mean IoU after trim on the SFT test split (260 examples at the 100-piece cap: 188 procedural, 72 real meshes, of which 30 have a teacher build covering under half the target), for the learned stop and for masked stop. Made before any benchmark run of the checkpoint.

checkpoint	STOP	all	procedural	real	real, weak teacher
greedy teacher	–	0.843	0.943	0.584	–
SFT v1	learned / masked	0.770 / 0.775	0.863 / 0.864	0.527 / 0.543	0.175 / 0.184
SFT v2	learned / masked	0.766 / 0.782	0.870 / 0.876	0.493 / 0.535	0.123 / 0.179
RL v1	learned / masked	0.824 / 0.835	0.936 / 0.936	0.531 / 0.569	0.190 / 0.206
RL v2, seed 0	learned / masked	0.828 / 0.840	0.937 / 0.939	0.542 / 0.581	0.147 / 0.208
RL v2, seed 1	learned / masked	0.821 / 0.835	0.931 / 0.935	0.533 / 0.574	0.141 / 0.204
RL v2, no stability reward	learned / masked	0.834 / 0.842	0.940 / 0.941	0.559 / 0.587	0.188 / 0.223
RL v2, no connectivity inputs	learned / masked	0.766 / 0.805	0.885 / 0.906	0.455 / 0.543	0.129 / 0.197

D Stability model

Bricks are rigid and loaded by their own weight (2.32 g per 2×4 brick, scaled by footprint area). Each shared stud cell between an upper and a lower brick carries vertical force at four sub-points near the cell corners; ground cells carry compression only. Per brick, vertical force balance and torque balance about x and y are imposed. A linear program finds the smallest tension utilisation t such that every joint force lies in $[-tT/4, +\infty)$ with $T = 0.98\,\mathrm{N}$ per stud cell. StableLego [22] measured 0.98 N as the capacity of one contact point (three to four per stud), so dividing it over four points per stud makes this model about four times more conservative than theirs; we keep the conservative reading because no capacity was measured in this project. StableLego also moves the equilibrium residuals into the objective so that every structure is solvable, whereas this program keeps equilibrium as a hard constraint and reports infeasibility as a topple. $t \leq 1$ is reported as stable under the model; an infeasible program even with unlimited tension means the centre of mass has left the support (topple). The model has no lateral loads, shear, friction, deformation, hand forces or dynamics, and its constants were not measured on any physical brick in this project. One program takes about 20 ms for a 75-piece build.

E Assembly planner

Rules: one brick per step, inserted straight down onto a horizontal work surface, starting from a ground brick. A brick may be added when it does not collide, attaches to the assembled component (ground contact or studs of a placed brick directly below), its vertical insertion path is clear, and it does not seal an unplaced brick beneath it. The last rule is an exact dead-end test, which makes greedy selection complete: if any full order exists, the first unplaced brick of that order is always attachable and safe, so the planner never backtracks. Ties break on lowest layer, then most placed supporters, then id, which gives a layer-by-layer order. For this catalogue (same-height upright bricks, every non-ground brick supported from below) every structurally valid build has a planned order; the validator replays the steps and checks the insertion path.

F Reproducibility

All code, benchmark records, audit logs and figure scripts are in the repository. The commands below regenerate the paper's numbers from a fresh checkout (training needs the Thingi10K download, about 1.5 GB, and the laptop GPU):

```
make bench                                  # frozen greedy baseline + audit
make sft-train sft-bench rl-train rl-bench  # v1 ladder
make data-v2 sft-train-v2 rl-train-v2 \
     rl-bench-v2                            # v2 ladder, seed 0
make v2-seed1 rl-bench-v2-seeds             # v2 ladder, seed 1
make rl-ablations rl-bench-v2-ablations     # ablations
make paper-builds paper-figs paper-check \
     paper-pdf                              # figures, macros, checks, this PDF
```

Every number in the main tables is a macro written by the figure script from the benchmark records, so the text cannot drift from the data. Decoding runs on CPU and is bit-identical across processes; voxelization pins the random state used by the containment test. The benchmark files are the six `.jsonl` files under `docs/benchmarks/` named in Section 4.

G Per-case results

Per-case IoU after trim at the 200-piece cap (0 for a missing or invalid build). RL columns use the masked stop; cells is the number of occupied target cells out of 2048 (512 for the cube).

case	source	cells	search	SFT v2	RL v2 s0	RL v2 s1	no stab.	no conn.
bridge	procedural	224	0.964	0.964	0.964	0.964	0.964	0.964
cube	procedural	256	1.000	0.641	1.000	1.000	1.000	1.000
cylinder	procedural	608	1.000	1.000	1.000	1.000	1.000	1.000
flat_sheet_impossible	procedural	160	0.050	0.050	0.050	0.050	0.050	0.050
l_shape	procedural	540	1.000	1.000	1.000	1.000	1.000	1.000
pyramid	procedural	336	0.946	0.911	0.946	0.928	0.918	0.911
slab	procedural	480	1.000	1.000	1.000	1.000	1.000	1.000
sphere	procedural	392	0.975	0.959	0.957	0.954	0.959	0.964
t_shape	procedural	288	1.000	1.000	1.000	1.000	1.000	1.000
thin_pole	procedural	0	0.000	0.000	0.000	0.000	0.000	0.000
tower	procedural	32	1.000	1.000	1.000	1.000	1.000	1.000
wedge	procedural	192	1.000	0.990	0.995	0.995	0.990	0.995
T100070	thingi10k	1070	1.000	0.938	1.000	1.000	0.999	0.998
T100894	thingi10k	84	0.913	0.905	0.907	0.905	0.907	0.907
T105120	thingi10k	56	0.200	0.267	0.194	0.271	0.194	0.138
T108578	thingi10k	68	0.057	0.000	0.057	0.057	0.057	0.057
T108884	thingi10k	276	0.953	0.931	0.933	0.953	0.971	0.932
T110201	thingi10k	32	0.250	0.250	0.250	0.273	0.250	0.250
T111878	thingi10k	60	0.109	0.109	0.109	0.109	0.109	0.108
T1130077	thingi10k	205	0.698	0.718	0.718	0.718	0.718	0.718
T113134	thingi10k	108	0.404	0.393	0.460	0.327	0.414	0.447
T130960	thingi10k	184	0.967	0.968	0.968	0.947	0.957	0.968
T1335002	thingi10k	400	0.980	0.946	0.950	0.947	0.941	0.948
T1417953	thingi10k	256	1.000	1.000	1.000	1.000	1.000	1.000
T144106	thingi10k	64	0.300	0.210	0.308	0.316	0.316	0.333
T1514900	thingi10k	92	0.706	0.593	0.652	0.702	0.652	0.626
T200212	thingi10k	365	0.989	0.981	0.992	0.986	0.984	0.981
T225935	thingi10k	290	0.907	0.852	0.883	0.865	0.888	0.837
T265728	thingi10k	49	0.648	0.571	0.588	0.588	0.588	0.635
T39108	thingi10k	128	0.800	0.420	0.448	0.420	0.250	0.780
T39922	thingi10k	168	0.667	0.964	0.667	0.667	0.661	0.667
T40447	thingi10k	452	1.000	1.000	1.000	1.000	1.000	1.000
T42025	thingi10k	84	0.024	0.023	0.023	0.035	0.023	0.023
T42035	thingi10k	154	0.987	0.808	0.957	0.727	0.807	0.662
T48124	thingi10k	37	0.295	0.295	0.311	0.295	0.311	0.311
T53749	thingi10k	278	0.458	0.439	0.449	0.432	0.387	0.449
T54820	thingi10k	82	0.427	0.427	0.427	0.546	0.427	0.427
T56630	thingi10k	359	0.955	0.978	0.978	0.975	0.978	0.943
T567026	thingi10k	20	0.200	0.000	0.200	0.200	0.200	0.200
T61192	thingi10k	26	1.000	1.000	1.000	1.000	1.000	1.000
T69537	thingi10k	64	0.812	0.778	0.788	0.866	0.778	0.778
T70559	thingi10k	13	0.231	0.231	0.231	0.231	0.231	0.200
T71307	thingi10k	76	0.237	0.224	0.272	0.350	0.350	0.330
T75090	thingi10k	238	0.790	0.701	0.793	0.810	0.778	0.800
T75651	thingi10k	125	0.920	0.921	0.931	0.936	0.929	0.929
T80597	thingi10k	188	0.883	0.819	0.851	0.856	0.841	0.815
T81589	thingi10k	212	0.972	0.972	0.972	0.967	0.963	0.968
T87521	thingi10k	25	1.000	0.833	0.833	1.000	0.926	0.926
T91350	thingi10k	433	0.993	0.984	0.986	0.989	0.973	0.984
T91452	thingi10k	42	0.159	0.159	0.159	0.109	0.159	0.159
T93881	thingi10k	91	1.000	1.000	1.000	1.000	1.000	1.000
T966906	thingi10k	265	0.817	0.814	0.879	0.773	0.896	0.782
mean			0.705	0.672	0.693	0.693	0.686	0.690